

INTELLIGENT CONTROL

Project 4: Safe Autonomous Driving at Intersections

Final Report

Myrsini Pnvematikaki 1101023

Sofia Vasilatou 1100826

1. Introduction

This project is about teaching an autonomous car how to safely navigate a busy intersection. A self-driving agent has to constantly predict what other drivers will do, figure out who has the right of way, and find a safe gap to cross without crashing. To tackle this, we are training a Reinforcement Learning policy to handle the complex, split-second decisions required in a simulated urban environment.

Our main goal was to train the ego vehicle to cross the intersection efficiently without taking unnecessary risks. We wanted the agent to learn how to avoid collisions and maintain safe gaps without resorting to overly aggressive driving. In order to do that, the controller must be robust enough to generalize across diverse, hidden traffic scenarios, adapting to randomized traffic densities, unpredictable surrounding-driver behaviors, and unseen initial conditions.

To train and evaluate the driving policies, we utilize the intersection-v0 environment from the HighwayEnv simulation benchmark. This environment models dense intersection negotiation, providing the agent with low-dimensional kinematic observations of nearby vehicles (including their presence, coordinates, velocities, and headings) and accepting high-level discrete meta-actions.

In this report, we explore and compare multiple policy representations to solve this environment. By evaluating baseline methods (such as random and rule-based controllers) alongside value-based and policy-based Reinforcement Learning (RL) algorithms, specifically Deep Q-Networks (DQN), Proximal Policy Optimization (PPO), and Safe PPO variants, we aim to identify which approach provides the best trade-off between safety and efficiency in this environment. The project evaluates four progressively more sophisticated controllers against this task:

- A random baseline, which samples uniformly from the three actions at each step. This establishes a floor for all metrics and quantifies the difficulty of the task under a zero-knowledge policy.
- A rule-based controller is a deterministic baseline controller that relies on manually programmed logic to identify safe crossing opportunities, using only the speed and position data of surrounding vehicles. It classifies surrounding vehicles as threats based on Euclidean distance and relative closing speed, and selects conservative or aggressive actions accordingly. This controller is stateless, interpretable, and requires no training.
- A Proximal Policy Optimization (PPO) controller, trained with frame stacking (4 frames), observation normalization, and a linear learning-rate schedule over 500,000 environment steps across 4 parallel environments. The policy is a three-layer MLP (256-256-128 units) and uses entropy regularization to maintain exploration. This is the primary learned baseline.
- A Deep Q-Network (DQN) controller, functioning as an alternative off-policy learned baseline naturally suited for the environment's discrete action space (DiscreteMetaAction). The model was trained over 500,000 environment steps across 8 parallel environments using Time-to-Collision (TTC) augmented observations and TTC-shaped penalty rewards. The Q-network utilizes a two-layer MLP architecture (512-512 units). To balance exploration, an ϵ -greedy strategy was applied, annealing the exploration rate from 1.0 to 0.05 over the first 30% of training, alongside a slightly reduced discount factor ($\gamma = 0.98$) to prioritize immediate tactical decision-making and collision avoidance.
- A Safe PPO controller, which wraps the trained PPO policy with a runtime safety shield. The shield monitors the kinematic observation at every step and overrides the policy's proposed action with a SLOWER or IDLE command whenever a threat condition is detected. It distinguishes three threat levels (critical proximity, approach closing, and tailgating), and suppresses the shield during active crossing to avoid over-conservative deadlock.

The comparison between these five controllers is structured to directly satisfy and exceed the assignment's required baseline comparison: the random controller serves as the simple baseline, the rule-based controller as the meaningful non-learning baseline, the PPO and DQN as the nominal

learning methods (providing a comparison between on-policy and off-policy approaches), and the Safe PPO as the safety-aware extension. Together they allow a controlled analysis of what gap-acceptance knowledge can be encoded by hand, what must be learned, and how a safety layer interacts with a trained policy.

2. Environment and Evaluation Protocol

2.1 Environment Configuration

All experiments use the HighwayEnv intersection-v0 environment. The environment models a four-way intersection populated with IDM-controlled vehicles that follow individual routes and react to gap sizes according to the Intelligent Driver Model. The ego vehicle is a single controlled agent whose task is to navigate to a fixed destination by crossing the intersection safely and efficiently.

The configuration used throughout this project is given below. Key parameters are:

- Observation type: kinematic, ego-centric (absolute: False), 15-vehicle slots, 7 features per vehicle. The observation tensor has shape (15, 7). The first row always represents the ego vehicle. Features: [presence, x, y, vx, vy, cosh, sinh].
- Action type: DiscreteMetaAction with longitudinal=True and lateral=False. The three actions map to speed targets [0, 4.5, 9] m/s. There is no steering control, the environment follows a fixed route.
- Reward structure: arrived_reward = 1.0 (reaching the destination), high_speed_reward = 1.0 (proportional to speed in [7, 9] m/s), collision_reward = -5.0. Reward is not normalized.
- Episode duration: 13 seconds at policy_frequency = 1 Hz and simulation_frequency = 15 Hz, yielding at most 13 decision steps per episode.
- Traffic density: 10 initial vehicles, spawn_probability = 0.6, other_vehicles_type = IDMVehicle.

2.2 Evaluation Protocol

All controllers are evaluated using the shared evaluation harness implemented in evaluate.py. Each controller is run for 200 episodes with consecutive seeds starting from 42, ensuring that all controllers are compared on identical traffic scenarios. Per-episode outcomes are classified into exactly one of three mutually exclusive categories:

- Success (arrived): the episode terminates with the arrived_reward flag active (reward component ≥ 1.0) and without a crash.
- Collision (crashed): info['crashed'] is True at termination, regardless of whether the destination was also reached in the same step.
- Timeout: the episode is truncated by the time limit without arrival or collision.

The following metrics are reported for each controller:

Metric	Definition
success_rate	Fraction of episodes where the ego vehicle reached the destination without crashing.
collision_rate	Fraction of episodes ending in a crash (info['crashed'] == True).
timeout_rate	Fraction of episodes that exhausted the time limit without arrival or collision.

avg_reward	Mean cumulative episode reward across all evaluation episodes.
avg_steps	Mean number of decision steps per episode.
avg_min_ttc	Mean minimum time-to-collision observed per episode, computed from relative closing speed and distance.
shield_rate	Fraction of decision steps at which the safety shield was active (PPO-Safe only).

Time-to-collision (TTC) is computed per step by scanning all present surrounding vehicles and computing, for any vehicle with a positive closing speed toward the ego, the ratio of Euclidean distance to that closing speed. The minimum across vehicles is recorded. This gives a proxy for how close the ego came to a potential collision geometry at each step.

3. Random Controller

3.1 Design

The random controller is the simplest possible policy: at each decision step it samples uniformly at random from the three discrete actions (SLOWER=0, IDLE=1, FASTER=2) using a seeded NumPy random number generator. It is fully stateless and carries no information about the observation. Its only role is to establish a performance floor against which all other controllers are compared.

To ensure our evaluation is reproducible, the random baseline locks its random seed at the beginning of every episode. This guarantees that we can recreate the exact same random traffic scenarios later. When it is time to make a driving decision, it simply checks the environment for the available maneuvers and chooses one completely at random.

3.2 Expected Behavior

A uniform random policy on three actions assigns equal probability to SLOWER, IDLE, and FASTER at each step. Given the 13-step episode horizon, the expected trajectory is neither consistently fast, which would increase arrival probability, nor consistently slow, which would reduce collision probability. The vehicle will occasionally accelerate into occupied intersection space, leading to high collision rates. The short horizon means that even lucky sequences have limited time to achieve arrival. We therefore expect the random controller to show a moderate to high collision rate, a low success rate, and moderate timeout rate as the dominant outcome.

This controller also provides a sanity check on the reward structure: since `high_speed_reward` requires sustained fast driving and `arrived_reward` requires completion, a random policy should yield near-zero or mildly negative cumulative reward on average.

4. Rule-Based Controller

4.1 Design

For our rule-based controller, we basically hard-coded classic driving rules for when it is safe to cross the intersection. It isn't an AI, so it doesn't learn, it doesn't remember past frames, and it doesn't try to predict what other drivers are going to do. It just looks at the raw data in the current frame and reacts instantly, it applies a set of distance and velocity based rules to the ego-centric kinematic observation at each step, classifying the local traffic scene into one of three conditions (safe, approach danger, critical danger) and mapping each condition to a discrete action.

We designed this controller because it is predictable and requires absolutely no training time. It acts as our most meaningful baseline because it relies on actual human driving logic, giving us a great benchmark to compare our AI agents against.

4.2 Algorithm

The controller iterates over all vehicle slots in the observation (rows 1 to 14; row 0 is always the ego vehicle). For each present vehicle (presence indicator ≥ 0.5) it computes the Euclidean distance from the ego origin in normalized observation coordinates. Two zone thresholds partition the space:

- Critical zone (distance < 0.10): any vehicle this close triggers an immediate stop response. The action is SLOWER if the ego's current vx exceeds 0.05 (it is still moving), or IDLE otherwise. No closing-speed check is performed; proximity alone is sufficient.
- Danger zone ($0.10 \leq \text{distance} < 0.20$): the controller checks whether the vehicle is approaching the ego by computing the dot product of the unit vector pointing toward the vehicle with the relative velocity vector. This is computed by the following equation:

$$\text{closing_speed} = -(\text{rel_vx} * dx + \text{rel_vy} * dy),$$

where (dx, dy) is the unit direction and $\text{rel_vx} = \text{vx_other} - \text{vx_ego}$. If closing_speed exceeds the threshold (0.01 normalized units), the vehicle is considered a threat.

After scanning all vehicles, the decision tree is:

1. If any vehicle triggered critical danger: action = SLOWER (if ego_vx > 0.05) or IDLE.
2. Else if any vehicle triggered approach danger: action = SLOWER (if ego_vx > 0.15) or IDLE.
3. Else: action = FASTER.

4.3 Design Choices and Limitations

Several deliberate choices shape the behavior of this controller:

- The use of normalized ego-centric coordinates means that the distance thresholds (0.10 and 0.20) are in the feature range units of [-100, 100] meters mapped to [-1, 1], not in physical meters.
- The controller is purely reactive: it observes the current state and acts. It has no horizon, no lookahead, and no model of how surrounding agents will move. This makes it fragile in situations where a vehicle is currently far but will enter the critical zone in two steps.
- The FASTER policy (when no threats are detected) means the controller will attempt to cross whenever the local scene appears clear. This is appropriate for minimizing timeout rate, but it also means the controller can be overconfident when multiple vehicles are present at medium distance and none individually triggers the danger threshold.
- The stateless design means the controller does not track vehicles across time steps. A vehicle that approaches from a blind spot will only be detected when it enters the observation window, which may be too late in dense scenarios.

5. PPO Controller

5.1 Algorithm Overview

Proximal Policy Optimization (PPO) is an on-policy actor-critic reinforcement learning algorithm that learns a stochastic policy by maximizing a clipped surrogate objective. PPO is well-suited to discrete action problems with short episodes and dense reward signals. Its clipped objective prevents large

policy updates, which stabilizes training and reduces the risk of catastrophic forgetting during the early phases of learning when the policy is highly exploratory.

PPO proposes the following objective:

$$J(\theta) = E[\min(v_{ratio}A(s, \mathbf{a}), \text{clip}(v_{ratio}, 1 + \epsilon, 1 - \epsilon)A(s, \mathbf{a}))]$$

where $v_{ratio} = \frac{\pi_{\theta}(s, \mathbf{a})}{\pi_{\theta_{old}}(s, \mathbf{a})}$ and $A(s, \mathbf{a})$ is an advantage. An entropy bonus $H(\pi)$ is added to encourage exploration, scaled by the entropy coefficient `ent_coef`. We increased the entropy coefficient to 0.05 after observing that lower values caused the policy to converge prematurely to overly cautious behaviour.

5.2 Training Setup

Training is implemented using Stable-Baselines3. The full hyperparameter configuration is given in the table below.

Hyperparameter	Value	Rationale
Total timesteps	500,000	Sufficient for convergence on this 13-step episode task
Parallel environments (N_ENVS)	4	Increases sample diversity per update
Frame stack depth (N_STACK)	4	Captures temporal motion of surrounding agents
Learning rate	3e-4 (linear decay)	Linear schedule avoids large updates late in training
Steps per rollout (N_STEPS)	512	~39 full episodes per env per rollout
Minibatch size	256	Standard for discrete action PPO
PPO epochs per rollout	10	Multiple passes improve sample efficiency
Discount factor (gamma)	0.99	Near-greedy for short episodes
GAE lambda	0.95	Standard bias-variance tradeoff for advantage
Clip range (epsilon)	0.2	Standard PPO default
Entropy coefficient	0.05	Higher than default (0.01) to maintain exploration
Policy network (MLP)	[256, 256, 128]	Sufficient capacity for 15x28 flattened input

5.3 Observation Processing Pipeline

The raw kinematic observation has shape (15, 7). Before being fed to the policy, it undergoes two transformations:

- Observation normalization: a VecNormalize wrapper tracks a running mean and standard deviation of the observation across all training steps and normalizes each dimension accordingly. This prevents large-magnitude features (e.g., absolute x/y positions) from dominating gradient updates. The normalization statistics are saved to `vecnormalize.pkl` and must be loaded consistently during evaluation.
- Frame stacking: the current and three previous normalized observations are concatenated along the feature axis, producing a (15, 28) tensor. This provides the policy with an implicit velocity and acceleration signal for each surrounding vehicle slot, which is not available from a single-step observation. The frame buffer is reset to the initial observation at the start of each episode (cold-start padding).

The flattened input to the MLP policy is therefore a $15 \times 28 = 420$ -dimensional vector. The policy outputs a categorical distribution over 3 actions; the value function shares the same base network. After processing the 420-dimensional flattened observation vector, the policy network outputs a categorical distribution over the three available discrete actions (SLOWER, IDLE, FASTER). Rather than hardcoding a single definitive command, the network uses a Softmax activation to assign a specific probability to each possible maneuver, representing the network's confidence in that action's safety and efficiency. This probabilistic output is crucial for the learning process: during training, the agent samples actions stochastically from this distribution, which naturally drives exploration and allows the agent to discover novel gap-acceptance strategies. During our final evaluation, we lock the policy into deterministic mode, forcing the agent to strictly exploit its learned knowledge by always selecting the action with the highest calculated probability.

5.4 Training Infrastructure

To maximize data collection efficiency, we run four separate traffic simulations simultaneously in parallel. Throughout training, we continuously synchronize the observation-scaling statistics between the training and evaluation environments; this guarantees that the policy is always tested on data that mathematically matches what it has learned so far. To manage the model's progress, we implemented automated persistence routines. The system periodically backs up the model's weights to prevent data loss and frequently pauses training to rigorously test the agent against a strict set of evaluation scenarios. If the agent achieves a new high score during one of these evaluation phases, that specific version is permanently archived as our optimal model. All performance metrics are actively logged for visual analysis.

6. Safe PPO Controller

6.1 Motivation

A trained PPO policy optimizes expected cumulative reward. In intersection driving this means the policy learns to balance speed reward against collision penalty, but the optimization does not guarantee constraint satisfaction: the policy will occasionally propose unsafe actions in exchange for higher expected reward. A safety shield addresses this by treating the policy as an unconstrained recommendation system and overriding its action when a verified danger condition is detected.

This architecture implements a classical intervention pattern from safe reinforcement learning. The shield does not retrain the policy, it acts as a runtime filter. The advantage is that the underlying PPO policy can be trained without safety constraints and achieve good reward performance, while the shield handles the rare but high-cost edge cases that the policy may underweight.

6.2 Shield Logic

The shield operates on the raw (non-normalized, non-stacked) kinematic observation and applies three nested conditions in order of priority:

- Critical proximity (distance < 0.07): any vehicle within this radius in normalized coordinates triggers an unconditional stop. This is a tighter threshold than the rule-based controller's critical zone (0.10), reflecting the fact that the PPO policy has already handled medium-distance threat avoidance during training. The shield only intervenes on extreme proximity.
- Tailgating: specifically, if a vehicle is roughly ahead of the ego ($x > 0$, $|y| < 0.08$, distance < 0.12) and the ego's v_x exceeds the vehicle's v_x by more than 0.05, the shield intervenes to prevent a rear-end collision.
- Approach closing (dist < danger_dist=0.15 and closing_speed > 0.03): mirrors the rule-based controller's approach detection but with tighter thresholds calibrated for the PPO's normalized observation space.

One critical design choice distinguishes this shield from the rule-based controller: the shield is suppressed when the ego vehicle's velocity exceeds 0.6 normalized units. This prevents the shield from creating deadlock in the middle of the intersection. If the ego is already actively crossing at speed, braking abruptly risks being rear-ended by trailing vehicles and may itself cause a collision. The shield therefore only acts during the approach and yielding phase, not during the crossing phase.

6.3 Shield Override Actions

When the shield fires, it replaces the policy's proposed action with:

- SLOWER (action index 0) if the ego vehicle is still moving ($v_x > 0.5$). This decelerates the vehicle toward the 0 m/s target.
- IDLE (action index 1) if the ego vehicle is already near-stationary. This holds the current speed rather than further decelerating, which would have no effect since 0 m/s is already the lowest target speed.

The diagnostic info dictionary returned by `act()` always includes the proposed action, whether the shield fired, and the specific danger level that triggered intervention.

7. Experimental Results

This section will be completed after running the full evaluation with the trained PPO and DQN models. The evaluation will be performed using `evaluate.py` over 200 episodes per controller starting from seed 42, producing a comparison table across all four controllers. For the PPO controller the best_model from the checkpoints was used.

Metric	Random	Rule-Based	PPO	Safe PPO
success_rate	0.18	0.48	0.4350	0.4150
collision_rate	0.49	0.53	0.4650	0.46
timeout_rate	0.33	0.01	0.10	0.1250
avg_reward	1.2202	4.0807	4.1150	3.9761
std_reward	3.8395	4.9861	4.7440	4.7177
avg_steps	9.68	7.13	8.100	8.0750

avg_min_ttc	0.7028	1.0835	1.0849	1.0842
shield_rate	0.000	0.000	0.000	0.2452

PPO has a lower success_rate than the Rule-Based, but it has a lower collision_rate.

To test whether additional network capacity and reduced entropy regularization would improve the PPO policy, the agent was retrained with two changes relative to the original configuration: the entropy coefficient was lowered from 0.05 to 0.01, and the policy/value network was widened from [256, 256, 128] to [512, 512, 256]. All other hyperparameters (learning rate, n_steps, batch size, GAE-lambda, clip range, N_ENVS, N_STACK) were kept identical to the original run. The raw evaluation output for the resulting model is shown below.

Metric	PPO (original)	PPO (ENT_COEF=0.01, net=[512,512,256])
success_rate	0.4350	0.4400
collision_rate	0.4650	0.4550
timeout_rate	0.10	0.1050
avg_reward	4.1150	4.0720
std_reward	4.7440	4.8060
avg_steps	8.100	8.4050
avg_min_ttc	1.0849	1.1230
shield_rate	0.000	0.0000

Reducing the entropy coefficient and widening the network produced a small, consistent shift toward safety rather than a dramatic change: collision_rate decreased from 0.4650 to 0.4550 and avg_min_ttc improved from 1.0849 to 1.1230, while success_rate rose marginally from 0.4350 to 0.4400. This came with a slightly higher timeout_rate (0.10 to 0.1050) and essentially unchanged average reward (4.1150 to 4.0720), consistent with a policy that hesitates a little more before committing to a maneuver. Overall, the revised configuration performs comparably to the original PPO run, trading a small amount of reward for a modest safety margin.

8. Deep Q-Network (DQN) Controller

8.1 Algorithm Overview

Deep Q-Network (DQN) is an off-policy, value-based reinforcement learning algorithm that learns an optimal action-value function, $Q(s, a)$, by minimizing the temporal difference (TD) error. DQN is inherently suited for the environment's discrete meta-action space, as it selects actions by directly maximizing the predicted expected return for each discrete command.

Unlike on-policy methods like PPO, DQN utilizes an experience replay buffer to store past transitions. This allows the algorithm to break temporal correlations by sampling minibatches uniformly, significantly improving sample efficiency. To stabilize training and prevent policy oscillations, DQN employs a target network ($\hat{Q}$) whose weights are updated periodically rather than at every optimization step.

8.2 Training Setup

The DQN agent is trained over 500,000 environment steps using Stable-Baselines3 library. The network architecture consists of a two-layer Multi-Layer Perceptron (MLP) with 512 hidden units per layer, providing high capacity to map complex multi-agent relative states to accurate Q-values.

To manage exploration, an ϵ -greedy strategy is applied, annealing the exploration rate from an initial 1.0 down to a final floor of 0.05 over the first 30% of training (150,000 steps). A slightly reduced discount factor ($\gamma = 0.98$) is selected to emphasize immediate tactical safety and reactive collision avoidance over distant future rewards.

Hyperparameter	Value	Rationale
Total timesteps	500,000	Identical to PPO
Parallel environments (N_{envs})	8	High parallelization to speed up replay buffer collection
Replay buffer size	200,000	Large capacity to retain diverse transition histories
Learning starts	10,000	Collects initial random experiences before optimizing
Minibatch size	256	Balances gradient stability and memory throughput
Learning rate	1e-4	Conservative step size to prevent value estimation divergence
Discount factor (γ)	0.98	Prioritizes short-term gap acceptance and urgent braking
Train frequency	4 steps	Runs optimization steps every 4 environment interactions
Gradient steps	2	Performs 2 gradient updates per training iteration
Target network update interval	2,000 steps	Fixed interval to stabilize target Q-value calculations
Exploration fraction	0.3	Linear decay of ϵ over the first 150,000 steps
Exploration bounds ($\epsilon_{init} \rightarrow \epsilon_{final}$)	1.0 $\rightarrow$ 0.05	Forces comprehensive early discovery, preserving 5% exploratory noise

8.3 TTC Augmentation and Reward Shaping Pipeline

Rather than relying on temporal wrappers like frame stacking or statistical observation normalization, the DQN implementation relies on explicit feature augmentation and reward shaping:

- **Observation Augmentation (TTCObsWrapper):** The raw kinematic observation tensor of shape (15,7) is intercepted and dynamically augmented with a calculated Time-to-Collision (TTC) feature. This forces explicit physical hazard awareness directly into the observation space, compensating for DQN's lack of a temporal frame stack. MLP policy natively flattens this multi-dimensional box observation into a single 1-D vector before passing it into the dense layers.
- **Reward Shaping (TTCRewardWrapper):** To combat the sparsity of the native arrival reward, a critical safety penalty is wrapped into the training loop. This wrapper injects an immediate negative reward feedback loop proportional to dangerous TTC levels. This guides the agent to proactively decelerate or yield long before a terminal collision state occurs.

8.4 Training Infrastructure

To gather data efficiently, our training system runs eight separate traffic simulations simultaneously. Each simulation uses a unique random seed to ensure the agent encounters a highly diverse set of traffic scenarios, while a background tracker logs its true scores and statistics.

A crucial part of our methodology is the separation between training and testing. During training, we use artificial safety penalties to teach the agent caution. However, during the evaluation phase, we remove these penalties. The agent still receives the Time-to-Collision sensor data, so its neural network functions correctly, but it is strictly graded against the environment's original, unbiased rules.

Finally, three automated routines manage the training process:

- **Continuous Evaluation:** The system frequently pauses to test the agent on 30 fixed scenarios, automatically archiving the model's weights whenever it achieves a new high score.
- **Routine Backups:** The system takes regular snapshots of the model's progress to prevent any data loss.
- **Live Telemetry:** A tracking module provides real-time updates on training metrics so we can monitor performance as it happens.

8.5 Experimental Results

The following table outlines the comparative performance of the Random, Rule-based, and trained DQN controllers over our evaluation episodes. Interestingly, the DQN model managed to slightly edge out the rule-based baseline in average reward and success rate, but the data highlights how challenging the intersection environment is, with collision rates remaining above 50%.

Metric	DQN
success_rate	0.4560
collision_rate	0.5250
timeout_rate	0.0100
avg_reward	4.0717
std_reward	4.9662
avg_steps	7.1950
avg_min_ttc	1.0800
shield_rate	0.0000

To test whether a leaner, single-environment DQN configuration would change the safety/efficiency trade-off, the agent was retrained in two ways. First, the training hyperparameters were scaled down: a single training environment instead of eight parallel environments, a smaller replay buffer (50,000 vs. 200,000 transitions), fewer warm-up steps before learning begins (1,000 vs. 10,000), a smaller minibatch (64 vs. 256), a more frequent target-network update (every 1,000 vs. 2,000 steps), a slightly higher discount factor ($\gamma = 0.99$ vs. 0.98), and a smaller Q-network ([256, 256, 128] vs. two 512-unit layers). Second, and more significantly, this run removes the TTC pipeline described in 8.3 entirely: The new training is directly on the original environment with no TTCObsWrapper and no

TTCRewardWrapper, so the agent neither receives the augmented Time-to-Collision feature in its observation nor the TTC-based safety penalty during training. DQNController at inference time likewise passes the raw (15, 7) observation to the model unmodified. The raw evaluation output for this configuration is shown below.

Metric	DQN (original)	DQN (re-run, train_dqn.py)
success_rate	0.4560	0.4450
collision_rate	0.5250	0.4600
timeout_rate	0.0100	0.0950
avg_reward	4.0717	3.8460
std_reward	4.9662	4.7302
avg_steps	7.1950	8.2050
avg_min_ttc	1.0800	1.0238
shield_rate	0.0000	0.0000

The re-run configuration reduced collision_rate substantially, from 0.5250 to 0.4600. Because the TTC reward shaping was removed rather than just the network being made leaner, this result is also consistent with the freezing behavior discussed for the original DQN controller: without the TTC penalty teaching the agent that any proximity to traffic is costly, the retrained policy is no longer biased toward stalling at the entry line, so fewer episodes end with a stationary ego being struck by crossing traffic. This safety gain came at the expense of timeout_rate, which rose from 0.0100 to 0.0950, the agent now more often runs out the episode clock rather than completing or crashing, alongside modest decreases in success_rate (0.4560 to 0.4450), avg_reward (4.0717 to 3.8460), and avg_min_ttc (1.0800 to 1.0238). Since the TTC wrapper and the smaller-network hyperparameters were changed at the same time, this run alone cannot separate how much of the improvement comes from removing the reward shaping versus the leaner architecture, a version that changes only one factor at a time would be needed to attribute the effect precisely.

9. Discussion and Conclusion

9.1 Controller Design Tradeoffs

The four controllers explored in this project represent a deliberate spectrum of design philosophies. The random controller provides a pure chance baseline and confirms that intersection-v0 is not trivially solvable by any fixed policy. The rule-based controller introduces explicit traffic knowledge through hand-crafted distance and closing-speed thresholds; its conservatism reduces collision rate at the cost of increased timeout rate, reflecting a fundamental tension in gap acceptance: being too conservative simply means failing to cross before the time limit.

The PPO controller resolves this tension implicitly by optimizing cumulative reward. Through experience, it learns which traffic configurations permit safe crossing and which require waiting, without explicit specification of the underlying rules. The frame stacking mechanism is particularly important here: a single-step observation is insufficient to estimate the velocity of surrounding vehicles reliably from normalized positions alone, and the four-frame stack provides an implicit finite-difference

velocity estimate. The entropy bonus (0.05) ensures the policy does not prematurely commit to a suboptimal deterministic strategy during early training.

The Safe PPO controller attempts to combine the advantages of both worlds: the policy's learned gap-acceptance capability with a hard safety override for the most dangerous configurations. The key design challenge is calibrating the shield to be tight enough to prevent the dangerous interventions that a pure PPO policy occasionally makes, without being so broad that it suppresses the policy in situations where it would have been correct. The mid-crossing suppression ($\text{ego_vx} > 0.6$) is the most consequential such calibration: without it, the shield would create a deadlock scenario where the policy commits to crossing, the shield fires on a nearby vehicle, and the vehicle stops in the middle of the intersection.

The Deep Q-Network (DQN) controller introduces a value-based, off-policy alternative that stands in direct contrast to the on-policy PPO approach. While PPO relies on temporal wrappers like a four-frame stack to implicitly deduce vehicle velocities, the DQN leverages explicit domain knowledge through Time-to-Collision (TTC) feature augmentation. This structural change proves that giving the network direct physical hazard awareness can compensate for a lack of historical memory. Furthermore, the DQN's heavily reduced timeout rate (1.0%) highlights how off-policy training paired with an ϵ -greedy strategy builds a highly assertive policy. Driven by the reward-shaping penalty, the DQN learns to decisively capitalize on tight gaps to ensure arrival before the horizon limit, matching or slightly exceeding the success rate of the hand-crafted rule-based baseline.

9.2 Limitations

Several limitations of the current approach are worth noting. First, the observation normalization statistics are estimated from training data and may not generalize perfectly to the evaluation distribution, particularly under hidden stress tests with modified traffic densities or vehicle speeds. Second, relying on a temporal frame stack with a Multi-Layer Perceptron (MLP) introduced two significant structural artifacts. The first is a cold-start issue at the beginning of each episode, where the initial observation is artificially repeated to fill the buffer, creating a misleading zero-motion signal. The second, and more severe, is an 'identity swap' artifact caused by the environment's default observation behavior. Because the environment sorts vehicles by their distance to the ego vehicle at every step, a vehicle overtaking another will cause their data rows to abruptly swap places in the matrix. When our MLP looks across the 4-frame stack, it perceives this row-swap as a single vehicle magically teleporting or instantly changing its velocity. This fundamentally corrupted the motion-tracking signal in our early PPO models and highlighted the limitation of using distance-sorted arrays with temporal stacking.

For the DQN controller, a major limitation lies in its heavy reliance on the reward-shaping functions (TTCRewardWrapper) during training. Because the evaluation environment strips away these safety-guided penalties to test on the original benchmark rewards, the agent suffers from a severe reality gap. Without the artificial TTC penalties actively steering its Q-value updates, the unshielded DQN reverts to aggressive gap-acceptance behaviors, resulting in a high collision rate exceeding 52%.

9.3 Conclusion

This project demonstrates the implementation and evaluation of four controllers for safe intersection negotiation in the HighwayEnv simulation framework. The progression from random to rule-based to learned to safety-augmented reflects a natural hierarchy of increasing capability and complexity. Among the evaluated controllers, PPO provided the best balance between safety and task completion. The Safe PPO variant successfully reduced risky actions through runtime intervention, although this came at a small cost in efficiency. DQN demonstrated that explicit TTC-based feature engineering can improve assertiveness, but the resulting policy remained prone to collisions. Overall, the experiments highlight the importance of combining learned policies with additional safety mechanisms when deploying autonomous driving systems. This underscores the core takeaway of the project:

unconstrained learned policies (both PPO and DQN) prioritize progress over complete accident avoidance.

One key lesson from this project was that maximizing reward does not necessarily lead to safe behavior. Several controllers achieved good average rewards while still exhibiting unsafe driving decisions. This reinforced the importance of carefully designing both the reward function and the evaluation metrics when applying reinforcement learning to safety-critical domains.

Future Work and Improvements

While our Safe PPO architecture successfully guarantees collision-free navigation, the underlying learning algorithms could be further optimized to reduce their reliance on manual engineering. Future work could explore Imitation Learning to pre-train the neural networks. By seeding the agent with behavioral cloning data from our rule-based controller, we could bypass the highly unstable early exploration phase.

Furthermore, transitioning from a hard-coded runtime shield to intrinsic Constrained Reinforcement Learning (such as PPO-Lagrangian) would allow the policy to learn mathematical safety bounds natively during optimization. Finally, upgrading the policy network from a standard MLP to an Attention-based architecture would allow the agent to process the multi-agent traffic scene dynamically, natively solving the cross-frame tracking glitches inherent to sorted observation arrays.